Day 10 Task :
Prompting AI Agents & Output Parsing

November 9, 2025

1 The Critical Role of Prompting AI Agents

The quality and consistency of an AI agent's performance are directly dependent on the quality of its **system prompt**. The prompt acts as the agent's "code" or "job description," telling it how to behave.

1.1 Reactive vs. Proactive Prompting (The Key Lesson)

A common mistake is to "proactively" write a very long, detailed prompt before ever testing the agent. This is difficult to debug and fails to account for real-world scenarios.

The more effective method is **Reactive Prompting**:

1. **Start with nothing:** Begin with an empty system prompt.
2. **Add one tool:** Connect a single tool (e.g., Google Calendar).
3. **Test and Observe:** Give the agent a task (e.g., "Schedule a meeting") and watch it fail.
4. **React and Fix:** Observe the specific error. For example, the agent might not know the current date. You then add *one single line* to the prompt to fix that specific error (e.g., "Here is the current date: `{{ $now }}`").
5. **Repeat:** Test again. Once it works, add the next tool. If it fails (e.g., it doesn't know *when* to use the tool), add another line to the prompt (e.g., "Use the `create_event` tool when a user asks to schedule something.").

This iterative, step-by-step process (Test $\rightarrow$ Reprompt $\rightarrow$ Test) ensures every line in your prompt has a specific purpose and makes the agent far more robust and easier to debug.

1.1.1 Live Prompting Example

- **Test 1:** User asks: "Create an event for 7 PM."
- **Failure:** Agent creates an event, but for a random, incorrect date (e.g., a date in 2023).
- **Reaction 1 (Fix):** Add to prompt: "Final Notes: Here is the current date and time: `{{ $now }}`".
- **Test 2:** User asks: "Send an email to Bob."
- **Failure:** Agent calls the `SendEmail` tool, but the email signs off with a generic placeholder like "[Your Name]".

- **Reaction 2 (Fix):** Add to prompt: ”Rules: Always sign off emails as Frank.”
- **Test 3:** User asks: ”Add a ’Promotion’ label to my last email from Nate.”
- **Failure:** The agent calls the `AddLabel` tool first and fails because it doesn’t have the `Message_ID` or the `Label_ID`.
- **Reaction 3 (Fix):** See ”Hard Prompting” example below.

1.2 Hard Prompting: Fixing Specific Failures

”Hard prompting” is a part of the reactive method. When an agent makes a logical error, you ”hard-code” a rule or example into the prompt to prevent it from happening again.

- **Example Failure:** You ask the agent to ”Email Bob,” but it tries to use the `SendEmail` tool without first finding Bob’s email address.
- **Example Fix (Hard Prompt):** You add a rule in the ”Tools” section: ”You must **always** use the `ContactDatabase` tool to get the email address *before* you use the `SendEmail` tool.”

1.3 Core Components of a System Prompt

Using the reactive method, you will gradually build a prompt that includes these key sections.

1.3.1 Example Combined System Prompt

```
You are a helpful assistant.

--- TOOLS ---
1. create_event: Use this to create a calendar event.
2. send_email: Use this to send an email.
   - Rule: Always sign off emails as Frank.
   - Rule: You must always use 'get_contacts' to find
     the email address *before* using this tool.
3. get_contacts: Use this to get a user's email address.

--- RULES ---
1. Be polite and concise.
2. If you don't know an answer, say so.

--- FINAL NOTES ---
Here is the current date and time: {{ $now }}
```

2 Output Parsing: Getting Structured Data

2.1 The Problem

By default, an AI agent outputs a single block of text. This is problematic for automation.

- **Example Bad Output:** ”Okay, here is the email subject: ‘Meeting Update‘ and here is the body: ‘Hi Bob...‘”
- **The Issue:** You cannot easily map this single text block to the separate ”Subject” and ”Body” fields in a ”Send Email” node.

2.2 The Solution: Structured Output Parser

Output Parsing forces the agent to respond in a specific format you define, most commonly JSON.

1. **How to Use:** In the n8n AI Agent node, enable the ”Require Specific Output Format” option. This adds an ”Output Parser” slot.
2. **Select Parser:** Connect a **Structured Output Parser** node.
3. **Define Schema:** Inside the parser, you provide a JSON schema (a template) of the exact format you want. You don’t need to write this by hand.
4. **Ask ChatGPT:** As shown in the video, you can simply ask ChatGPT: ”Help me write a JSON example for a structured output parser in n8n. I need a ‘subject‘ field and a ‘body‘ field.”
5. **Add Schema to Node:** Paste the JSON schema from ChatGPT into the parser node:

```
{
  "subject": "string",
  "body": "string"
}
```

2.3 The Result

Now, when the agent responds, its output is no longer a text block. It is a clean JSON object:

```
{
  "subject": "Meeting Update",
  "body": "Hi Bob..."
}
```

This allows you to easily use expressions in your ”Send Email” node to map the subject (`{{ $json.output.subject }}`) and body (`{{ $json.output.body }}`) correctly.